KUZCLAW - CYBER - VS01

This procedure documents the deployment of the KUZAI monitoring workflow for THE KUZ NETWORK.

The workflow is centered on the local control host, which orchestrates the full chain -->

- collect infrastructure and cybersecurity data from the remote VPS over SSH
- send the raw report to the model host over HTTP for KUZAI summary generation
- build the final plain text and HTML email on the local control host
- inject the final email through the VPS Postfix service over SSH
- run the workflow automatically every hour through systemd

The effective workflow is -->

LOCAL CONTROL HOST --> REMOTE VPS --> RAW DATA COLLECTION

LOCAL CONTROL HOST --> MODEL HOST --> KUZAI SUMMARY GENERATION

LOCAL CONTROL HOST --> REMOTE VPS POSTFIX --> EMAIL DELIVERY

INFRASTRUCTURE

- LOCAL CONTROL HOST--> `DEBIAN` - `$IP`
- MODEL HOST--> `$IP`
- REMOTE VPS--> `$IP`
- EMAIL RECIPIENT--> `admin@kuzai.org`

SOFTWARE STACK

LOCAL CONTROL HOST-->

- Debian
- OpenClaw
- systemd
- Python 3
- SSH client

MODEL HOST-->

- llama.cpp
- OpenAI-compatible API at `http-->//$IP-->8080/v1`

REMOTE VPS-->

- Debian 12
- Postfix

- Apache2
- Docker
- Fail2ban
- OpenSSH Server

WORKFLOW SYNOPTIC

```
DEBIAN $IP
  |
  |-- SSH --> VPS $IP
  |            |- system status
  |            |- SSH events
  |            |- Fail2ban counters
  |            |- Apache2 status + vhosts
  |            |- Docker status + table
  |            |- Postfix /usr/sbin/sendmail -t
  |
  |-- HTTP --> llama.cpp $IP-->8080/v1
  |            |- summary generation
  |
  |-- systemd timer (hourly)
               |- raw report
               |- KUZAI summary
               |- HTML email build
               |- email injection via VPS Postfix
```

SSH CONNECTIVITY

Create the local key on `DEBIAN`-->

```
mkdir -p /root/.ssh
chmod 700 /root/.ssh
ssh-keygen -t ed25519 -a 100 -f /root/.ssh/openclaw_vps_ed25519 -C "openclaw@$IP-to-$IP"
chmod 600 /root/.ssh/openclaw_vps_ed25519
chmod 644 /root/.ssh/openclaw_vps_ed25519.pub
```

Install the public key on the VPS root account.

Create the OpenClaw user SSH configuration on `DEBIAN`-->

```
install -d -m 700 -o openclaw -g openclaw /var/lib/openclaw/.ssh
install -m 600 -o openclaw -g openclaw /root/.ssh/openclaw_vps_ed25519
```

```
/var/lib/openclaw/.ssh/openclaw_vps_ed25519
install -m 644 -o openclaw -g openclaw /root/.ssh/openclaw_vps_ed25519.pub
/var/lib/openclaw/.ssh/openclaw_vps_ed25519.pub
cat > /var/lib/openclaw/.ssh/config <<'EOF_SSH'
Host vps-openclaw
    HostName $IP
    User root
    IdentityFile /var/lib/openclaw/.ssh/openclaw_vps_ed25519
    IdentitiesOnly yes
EOF_SSH
chown openclaw-->openclaw /var/lib/openclaw/.ssh/config
chmod 600 /var/lib/openclaw/.ssh/config
```

Validation-->

```
su - openclaw -c "ssh vps-openclaw 'echo OPENCLAW_HOME_OK; hostnamectl --static; id'"
```

OPENCLAW SERVICE

Create `/etc/systemd/system/openclaw.service`-->

```
[Unit]
Description=OpenClaw hardened gateway
After=network-online.target
Wants=network-online.target

[Service]
User=openclaw
Group=openclaw
WorkingDirectory=/var/lib/openclaw
Environment=HOME=/var/lib/openclaw
Environment=PATH=/var/lib/openclaw/.npm-global/bin-->/usr/local/sbin-->/usr/local/bin--
>/usr/sbin-->/usr/bin-->/sbin-->/bin
EnvironmentFile=/etc/openclaw/openclaw.env
ExecStart=/var/lib/openclaw/.npm-global/bin/openclaw gateway --port 18789
Restart=always
RestartSec=3
NoNewPrivileges=yes
PrivateTmp=yes
PrivateDevices=yes
ProtectSystem=strict
```

```
ProtectHome=yes
ProtectControlGroups=yes
ProtectKernelTunables=yes
ProtectKernelModules=yes
ProtectKernelLogs=yes
RestrictSUIDSGID=yes
LockPersonality=yes
RestrictRealtime=yes
SystemCallArchitectures=native
CapabilityBoundingSet=
AmbientCapabilities=
IPAddressDeny=any
IPAddressAllow=127.0.0.1
IPAddressAllow=-->-->1
IPAddressAllow=$IP
ReadWritePaths=/var/lib/openclaw
UMask=0077

[Install]
WantedBy=multi-user.target
```

Create `/etc/openclaw/openclaw.env`-->

```
LLAMACPP_API_KEY=llamacpp-local
```

Reload and start-->

```
systemctl daemon-reload
systemctl enable --now openclaw.service
systemctl status openclaw --no-pager
```

OPENCLAW JSON MODEL CONFIG

Relevant model configuration in `/var/lib/openclaw/.openclaw/openclaw.json`-->

```
{
  "gateway"--> {
    "port"--> 18789
  },
  "agents"--> {
    "defaults"--> {
```

```
      "model"--> {
        "primary"--> "llamacpp/bartowski_Mistral-7B-Instruct-v0.3-GGUF_Mistral-7B-Instruct-v0.3-Q4_K_M.gguf"
      }
    }
  },
  "models"--> {
    "providers"--> {
      "llamacpp"--> {
        "baseUrl"--> "http-->//$IP-->8080/v1",
        "apiKey"--> "${LLAMACPP_API_KEY}",
        "models"--> [
          {
            "id"--> "bartowski_Mistral-7B-Instruct-v0.3-GGUF_Mistral-7B-Instruct-v0.3-Q4_K_M.gguf",
            "name"--> "Remote llama.cpp Mistral 7B"
          }
        ]
      }
    }
  }
}
```

RAW REPORT SCRIPT

Create `/var/lib/openclaw/bin/vps_state_raw.sh`-->

```
#!/usr/bin/env bash
set -Eeuo pipefail

SSH_TARGET="vps-openclaw"
REPORT_DIR="/var/lib/openclaw/reports"

mkdir -p "$REPORT_DIR"

ts_utc="$(date -u '+%Y-%m-%d %H-->%M-->%S UTC')"
report_file="$REPORT_DIR/vps-state-$(date -u '+%Y%m%d-%H%M%S').txt"

ssh_run() {
  ssh -o BatchMode=yes -o ConnectTimeout=10 "$SSH_TARGET" "$@"
}
```

```
host="$(ssh_run 'hostnamectl --static')"
os_line="$(ssh_run 'grep ^PRETTY_NAME= /etc/os-release | cut -d= -f2- | tr -d "\""' || true)"
kernel="$(ssh_run 'uname -r')"
uptime_line="$(ssh_run 'uptime')"

disk_root="$(ssh_run "df -h / | tail -n 1")"
inode_root="$(ssh_run "df -i / | tail -n 1")"
mem_line="$(ssh_run "free -h | awk '/^Mem-->/{print \$0}'")"
swap_line="$(ssh_run "free -h | awk '/^Swap-->/{print \$0}'")"

active_services="$(ssh_run "systemctl list-units --type=service --state=active --no-legend
2>/dev/null | wc -l | tr -d ' '")"
inactive_services="$(ssh_run "systemctl list-units --type=service --all --state=inactive --no-
legend 2>/dev/null | wc -l | tr -d ' '")"
failed_services_count="$(ssh_run "systemctl --failed --type=service --no-legend 2>/dev/null |
wc -l | tr -d ' '")"
failed_services_list="$(ssh_run "systemctl --failed --type=service --no-legend --plain
2>/dev/null | awk '{print \$1}'" || true)"
if [ -z "${failed_services_list-->-}" ]; then
  failed_services_list="none"
fi

certbot_state="$(ssh_run 'systemctl is-active certbot.service 2>/dev/null || true')"
if [ -z "${certbot_state-->-}" ]; then
  certbot_state="unknown"
fi

upgradable_raw="$(ssh_run "apt list --upgradable 2>/dev/null | sed '1d' || true")"
upgradable_count="$(printf '%s\n' "$upgradable_raw" | sed '/^\s*$/d' | wc -l | tr -d ' ')"

docker_version="$(ssh_run "docker version --format '{{.Server.Version}}' 2>/dev/null || echo
unavailable")"
docker_running="$(ssh_run "docker ps -q 2>/dev/null | wc -l | tr -d ' '")"
docker_total="$(ssh_run "docker ps -aq 2>/dev/null | wc -l | tr -d ' '")"
docker_stopped="$(ssh_run "docker ps -aq -f status=exited 2>/dev/null | wc -l | tr -d ' '")"
docker_restarting="$(ssh_run "docker ps -aq -f status=restarting 2>/dev/null | wc -l | tr -d '
'")"
docker_unhealthy="$(ssh_run "docker ps -a --format '{{.Names}} {{.Status}}' 2>/dev/null | grep
-Ei 'unhealthy|restarting|exited' || true")"
```

```
if [ -z "${docker_unhealthy-->-}" ]; then
  docker_unhealthy="none"
fi

docker_table="$(ssh_run "docker ps -a --format 'table
{{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}' 2>/dev/null || true")"
docker_disk_usage="$(ssh_run "docker system df 2>/dev/null || true")"

public_tcp="$(ssh_run "ss -ltnp | { head -n 1; tail -n +2 | grep -Ev '127\\.0\\.0\\.1-->|\\[--
>-->1\\]-->'; }" || true)"

ssh_posture="$(ssh_run "grep -R -n -E
'^(PermitRootLogin|PasswordAuthentication|PubkeyAuthentication|KbdInteractiveAuthentication|Ch
allengeResponseAuthentication)' /etc/ssh/sshd_config /etc/ssh/sshd_config.d/* 2>/dev/null ||
true")"

last_ssh_attempts="$(ssh_run "
  echo '[RECENT SSH JOURNAL EVENTS]'
  journalctl -u ssh -n 80 --no-pager 2>/dev/null | grep -Ei
'Accepted|Failed|Invalid|Connection from|authentication|Disconnected|error-->' | tail -n 20 ||
true
  echo
  echo '[RECENT FAILED SSH LOGINS]'
  lastb -a 2>/dev/null | head -n 20 || true
" || true)"
if [ -z "${last_ssh_attempts-->-}" ]; then
  last_ssh_attempts="none"
fi

banned_ips_count="$(ssh_run "fail2ban-client status sshd 2>/dev/null | awk -F--> '/Currently
banned/ {gsub(/ /, \"\", \$2); print \$2}'" || true)"
if [ -z "${banned_ips_count-->-}" ]; then
  banned_ips_count="0"
fi

apache_status="$(ssh_run "systemctl status apache2 --no-pager 2>/dev/null | sed -n '1,20p'" ||
true)"
apache_vhosts="$(ssh_run "apache2ctl -S 2>&1 | sed -n '1,120p'" || true)"

{
```

```
echo "RAW VPS REPORT"
echo "Date--> $ts_utc"
echo "Host--> $host"
echo "OS--> $os_line"
echo "Kernel--> $kernel"
echo
echo "=== GLOBAL STATUS ==="
echo "$uptime_line"
echo "$disk_root"
echo "$inode_root"
echo "$mem_line"
echo "$swap_line"
echo
echo "=== SERVICE COUNTS ==="
echo "Active services--> $active_services"
echo "Inactive services--> $inactive_services"
echo "Failed services--> $failed_services_count"
echo "Failed services list--> $failed_services_list"
echo
echo "=== CERTBOT ==="
echo "$certbot_state"
echo
echo "=== PACKAGE UPDATES ==="
echo "Upgradable packages--> $upgradable_count"
printf '%s\n' "$upgradable_raw"
echo
echo "=== SSH POSTURE ==="
printf '%s\n' "$ssh_posture"
echo
echo "=== LAST SSH ATTEMPTS ==="
printf '%s\n' "$last_ssh_attempts"
echo
echo "=== FAIL2BAN ==="
echo "Currently banned IPs--> $banned_ips_count"
echo
echo "=== APACHE2 STATUS ==="
printf '%s\n' "$apache_status"
echo
echo "=== APACHE2 VHOSTS ==="
printf '%s\n' "$apache_vhosts"
```

```
  echo
  echo "=== DOCKER STATUS ==="
  echo "Docker version--> $docker_version"
  echo "Running containers--> $docker_running"
  echo "Total containers--> $docker_total"
  echo "Stopped containers--> $docker_stopped"
  echo "Restarting containers--> $docker_restarting"
  echo "Containers in anomaly--> $docker_unhealthy"
  echo
  echo "=== DOCKER TABLE ==="
  printf '%s\n' "$docker_table"
  echo
  echo "=== DOCKER DISK USAGE ==="
  printf '%s\n' "$docker_disk_usage"
  echo
  echo "=== PUBLIC LISTENING PORTS ==="
  printf '%s\n' "$public_tcp"
} > "$report_file"

echo "RAW_REPORT=$report_file"
```

Apply permissions-->

```
chown openclaw-->openclaw /var/lib/openclaw/bin/vps_state_raw.sh
chmod 750 /var/lib/openclaw/bin/vps_state_raw.sh
```

Validation-->

```
su - openclaw -c "/var/lib/openclaw/bin/vps_state_raw.sh"
latest_report="$(ls -1t /var/lib/openclaw/reports/vps-state-*.txt | head -n1)"
sed -n '1,260p' "$latest_report"
```

KUZAI SUMMARY SCRIPT

Create `/var/lib/openclaw/bin/test_vps_summary.py`-->

```
#!/usr/bin/env python3
import json
import os
from pathlib import Path
from urllib.request import Request, urlopen
```

```
from urllib.error import URLError, HTTPError

ENV_FILE = Path("/etc/openclaw/openclaw.env")
REPORT_DIR = Path("/var/lib/openclaw/reports")
MODEL_URL = "http-->//$IP-->8080/v1/chat/completions"
MODEL_NAME = "bartowski_Mistral-7B-Instruct-v0.3-GGUF_Mistral-7B-Instruct-v0.3-Q4_K_M.gguf"

def load_env_file(path--> Path) -> dict-->
    env = {}
    if not path.exists()-->
        return env
    for line in path.read_text(encoding="utf-8").splitlines()-->
        line = line.strip()
        if not line or line.startswith("#") or "=" not in line-->
            continue
        k, v = line.split("=", 1)
        env[k.strip()] = v.strip()
    return env

def main() -> int-->
    env = load_env_file(ENV_FILE)
    api_key = env.get("LLAMACPP_API_KEY") or os.environ.get("LLAMACPP_API_KEY")
    if not api_key-->
        print("ERROR--> LLAMACPP_API_KEY not found", flush=True)
        return 1

    reports = sorted(REPORT_DIR.glob("vps-state-*.txt"), key=lambda p--> p.stat().st_mtime,
reverse=True)
    if not reports-->
        print("ERROR--> no raw report vps-state-*.txt found", flush=True)
        return 2

    latest_report = reports[0]
    report_text = latest_report.read_text(encoding="utf-8")

    payload = {
        "model"--> MODEL_NAME,
        "temperature"--> 0.25,
        "messages"--> [
            {
```

```
                "role"--> "system",
                "content"--> (
                    "You are KUZAI, the local and independent intelligence of THE KUZ NETWORK. "
                    "Your mission is to monitor infrastructure health, systems stability, service exposure, "
                    "Docker workloads, Apache status, SSH activity, and cybersecurity posture. "
                    "You speak as an operational cyber-defense intelligence, not as a generic assistant. "
                    "You write in English only. Never use French. "
                    "All section titles must be uppercase. "
                    "The introduction must explicitly present KUZAI as the local and independent AI responsible "
                    "for the infrastructure and cybersecurity monitoring of THE KUZ NETWORK. "
                    "The introduction must be more elaborate, more personalized, and more narrative than before, "
                    "while remaining technical, sober, and factual. "
                    "Do not mention tools. Do not say you are just an assistant. Do not use marketing language. "
                    "You must interpret the data, prioritize issues, and explain why they matter. "
                    "You must use the newly available metrics--> service counts, last SSH attempts, current banned IP count, "
                    "Apache status, Apache vhosts, Docker status, and Docker table. "
                    "Use exactly this structure-->\n\n"
                    "KUZAI INTRODUCTION\n"
                    "<one personalized operational introduction in 5 to 8 sentences>\n\n"
                    "GLOBAL STATUS\n"
                    "<one concise paragraph>\n\n"
                    "INFRASTRUCTURE ANALYSIS\n"
                    "- ...\n"
                    "- ...\n"
                    "- ...\n\n"
                    "CYBERSECURITY ANALYSIS\n"
                    "- ...\n"
                    "- ...\n"
                    "- ...\n\n"
                    "SERVICE HEALTH\n"
                    "- ...\n"
```

```
"- ...\n"
"- ...\n\n"
"APACHE STATUS\n"
"- ...\n"
"- ...\n"
"- ...\n\n"
"DOCKER STATUS\n"
"- ...\n"
"- ...\n"
"- ...\n\n"
"WATCHPOINTS\n"
"- ...\n"
"- ...\n"
"- ...\n\n"
"RECOMMENDED ACTIONS\n"
"- ...\n"
"- ...\n"
"- ...\n\n"
"RAW TECHNICAL DATA\n"
"- Host--> ...\n"
"- OS--> ...\n"
"- Kernel--> ...\n"
"- Uptime--> ...\n"
"- Disk usage--> ...\n"
"- Memory usage--> ...\n"
"- Swap--> ...\n"
"- Active services--> ...\n"
"- Inactive services--> ...\n"
"- Failed services--> ...\n"
"- Certbot--> ...\n"
"- Upgradable packages--> ...\n"
"- SSH posture--> ...\n"
"- Last SSH attempts--> ...\n"
"- Currently banned IPs--> ...\n"
"- Apache status--> ...\n"
"- Apache vhosts--> ...\n"
"- Docker version--> ...\n"
"- Running containers--> ...\n"
"- Docker table--> ...\n"
"- Public listening ports--> ...\n\n"
```

```
                    "Rules-->\n"
                    "- English only\n"
                    "- UPPERCASE section titles only\n"
                    "- mention the real state of certbot\n"
                    "- mention the number of banned IPs\n"
                    "- mention the SSH exposure trend if visible in recent attempts\n"
                    "- mention Apache and vhost state if relevant\n"
                    "- mention Docker container state and exposure\n"
                    "- if something is healthy, say so briefly\n"
                    "- if something is risky, explain why\n"
                    "- do not invent values\n"
                    "- do not summarize raw data incorrectly\n"
                    "- keep the whole output readable in an email"
                ),
            },
            {
                "role"--> "user",
                "content"--> report_text,
            },
        ],
    }

    payload_file = REPORT_DIR / "vps-summary-payload.json"
    summary_file = REPORT_DIR / "vps-summary-latest.txt"

    payload_file.write_text(json.dumps(payload, ensure_ascii=False, indent=2), encoding="utf-
8")

    req = Request(
        MODEL_URL,
        data=json.dumps(payload).encode("utf-8"),
        headers={
            "Content-Type"--> "application/json",
            "Authorization"--> f"Bearer {api_key}",
        },
        method="POST",
    )

    try-->
        with urlopen(req, timeout=120) as resp-->
```

```
            body = resp.read().decode("utf-8")
    except HTTPError as e-->
        print(f"HTTP ERROR--> {e.code}", flush=True)
        print(e.read().decode("utf-8", errors="replace"), flush=True)
        return 3
    except URLError as e-->
        print(f"URL ERROR--> {e}", flush=True)
        return 4

    try-->
        data = json.loads(body)
        final_body = data["choices"][0]["message"]["content"].strip()
    except Exception as e-->
        print("JSON PARSE ERROR-->", flush=True)
        print(str(e), flush=True)
        print(body, flush=True)
        return 5

    summary_file.write_text(final_body + "\n", encoding="utf-8")
    payload_file.chmod(0o644)
    summary_file.chmod(0o644)

    try-->
        os.chown(payload_file, 999, 996)
        os.chown(summary_file, 999, 996)
    except PermissionError-->
        pass

    print(f"LATEST_REPORT={latest_report}")
    print(f"PAYLOAD_FILE={payload_file}")
    print(f"SUMMARY_FILE={summary_file}")
    print("=== SUMMARY CONTENT ===")
    print(final_body)
    return 0

if __name__ == "__main__"-->
    raise SystemExit(main())
```

Validation-->

```
python3 /var/lib/openclaw/bin/test_vps_summary.py
sed -n '1,260p' /var/lib/openclaw/reports/vps-summary-latest.txt
```

HTML MAIL BUILDER

Create `/var/lib/openclaw/bin/build_kuzai_mail.py`-->

```
#!/usr/bin/env python3
from __future__ import annotations

import html
import re
from datetime import datetime, timezone
from email.mime.multipart import MIMEMultipart
from email.mime.text import MIMEText
from pathlib import Path

MAIL_TO = "admin@kuzai.org"
SUMMARY_FILE = Path("/var/lib/openclaw/reports/vps-summary-latest.txt")
REPORT_DIR = Path("/var/lib/openclaw/reports")

def latest_raw_report() -> Path-->
    reports = sorted(REPORT_DIR.glob("vps-state-*.txt"), key=lambda p--> p.stat().st_mtime,
reverse=True)
    if not reports-->
        raise FileNotFoundError("No raw report found")
    return reports[0]

def strip_md(line--> str) -> str-->
    return line.strip().strip("*").strip()

def parse_summary_sections(summary--> str) -> dict[str, str]-->
    sections--> dict[str, str] = {}
    current = None
    buffer--> list[str] = []
    for raw in summary.splitlines()-->
        line = raw.rstrip()
        title = strip_md(line)
        if re.fullmatch(r"[A-Z][A-Z ]+[A-Z]", title)-->
            if current is not None-->
```

```
                sections[current] = "\n".join(buffer).strip()
            current = title
            buffer = []
            continue
        if current is not None-->
            buffer.append(line)
    if current is not None-->
        sections[current] = "\n".join(buffer).strip()
    return sections

def extract_raw_section(raw--> str, section_name--> str) -> str-->
    pattern = rf"^=== {re.escape(section_name)} ===\n(.*?)(?=^=== |\Z)"
    m = re.search(pattern, raw, re.S | re.M)
    return m.group(1).strip() if m else ""

def find_value(raw--> str, label--> str) -> str-->
    m = re.search(rf"^{re.escape(label)}-->\s*(.+)$", raw, re.M)
    return m.group(1).strip() if m else "N/A"

def extract_header_value(raw--> str, label--> str) -> str-->
    m = re.search(rf"^{re.escape(label)}-->\s*(.+)$", raw, re.M)
    return m.group(1).strip() if m else "N/A"

def bullets_to_html(text--> str) -> str-->
    lines = [line.strip() for line in text.splitlines() if line.strip()]
    if not lines-->
        return "<p>N/A</p>"
    bullet_lines = [line for line in lines if line.startswith("- ")]
    if bullet_lines and len(bullet_lines) == len(lines)-->
        items = "".join(f"<li>{html.escape(line[2-->])}</li>" for line in bullet_lines)
        return f'<ul class="bullet-list">{items}</ul>'
    paragraphs = "".join(f"<p>{html.escape(line)}</p>" for line in lines)
    return paragraphs

def parse_docker_table(raw_section--> str) -> list[list[str]]-->
    rows--> list[list[str]] = []
    lines = [line.rstrip() for line in raw_section.splitlines() if line.strip()]
    if not lines-->
        return rows
    for line in lines[1-->]-->
```

```
        cols = line.split("\t")
        if len(cols) < 4-->
            continue
        rows.append([cols[0].strip(), cols[1].strip(), cols[2].strip(), cols[3].strip()])
    return rows

def html_table(rows--> list[list[str]]) -> str-->
    if not rows-->
        return ""
    head = """
    <tr>
      <th>SERVICE</th>
      <th>IMAGE</th>
      <th>STATE</th>
      <th>PORTS</th>
    </tr>
    """
    body = []
    for service, image, state, ports in rows-->
        body.append(
            "<tr>"
            f"<td>{html.escape(service)}</td>"
            f"<td>{html.escape(image)}</td>"
            f"<td>{html.escape(state)}</td>"
            f"<td>{html.escape(ports)}</td>"
            "</tr>"
        )
    return f'<table class="data-table">{head}{"".join(body)}</table>'

def pre_box(text--> str) -> str-->
    if not text.strip()-->
        return "<p>N/A</p>"
    return f'<div class="raw-box"><pre>{html.escape(text.strip())}</pre></div>'

def extract_ssh_users(last_ssh_attempts--> str) -> list[str]-->
    users--> list[str] = []
    seen = set()
    for line in last_ssh_attempts.splitlines()-->
        line = line.strip()
        if not line-->
```

```
            continue
        patterns = [
            r"for invalid user ([A-Za-z0-9._-]+)",
            r"for ([A-Za-z0-9._-]+) from",
            r"authenticating user ([A-Za-z0-9._-]+)",
            r"Accepted publickey for ([A-Za-z0-9._-]+)",
            r"Accepted password for ([A-Za-z0-9._-]+)",
            r"Failed password for ([A-Za-z0-9._-]+)",
        ]
        matched = False
        for pat in patterns-->
            m = re.search(pat, line, re.I)
            if m-->
                user = m.group(1)
                if user not in seen-->
                    seen.add(user)
                    users.append(user)
                matched = True
                break
        if matched-->
            continue
        tokens = line.split()
        if tokens-->
            candidate = tokens[0]
            if (
                re.fullmatch(r"[A-Za-z0-9._-]+", candidate)
                and candidate.lower() not in {"btmp", "wtmp", "reboot"}
                and "ssh" in line.lower()
            )-->
                if candidate not in seen-->
                    seen.add(candidate)
                    users.append(candidate)
    return users[-->25]

def user_list_html(users--> list[str]) -> str-->
    if not users-->
        return '<p class="inline-users">No explicit SSH usernames extracted from recent
attempts.</p>'
    return '<p class="inline-users">' + " · ".join(html.escape(u) for u in users) + "</p>"
```

```
def build_html(summary_text--> str, raw_text--> str) -> str-->
    summary_sections = parse_summary_sections(summary_text)
    intro = summary_sections.get("KUZAI INTRODUCTION", "")
    global_status = summary_sections.get("GLOBAL STATUS", "")
    infra = summary_sections.get("INFRASTRUCTURE ANALYSIS", "")
    cyber = summary_sections.get("CYBERSECURITY ANALYSIS", "")
    service_health = summary_sections.get("SERVICE HEALTH", "")
    apache_status_summary = summary_sections.get("APACHE STATUS", "")
    docker_status_summary = summary_sections.get("DOCKER STATUS", "")
    watchpoints = summary_sections.get("WATCHPOINTS", "")
    actions = summary_sections.get("RECOMMENDED ACTIONS", "")
    host = extract_header_value(raw_text, "Host")
    service_counts = extract_raw_section(raw_text, "SERVICE COUNTS")
    last_ssh_attempts = extract_raw_section(raw_text, "LAST SSH ATTEMPTS")
    apache_status = extract_raw_section(raw_text, "APACHE2 STATUS")
    apache_vhosts = extract_raw_section(raw_text, "APACHE2 VHOSTS")
    docker_status = extract_raw_section(raw_text, "DOCKER STATUS")
    docker_table_raw = extract_raw_section(raw_text, "DOCKER TABLE")
    public_ports = extract_raw_section(raw_text, "PUBLIC LISTENING PORTS")
    fail2ban = extract_raw_section(raw_text, "FAIL2BAN")
    active_services = find_value(service_counts, "Active services")
    inactive_services = find_value(service_counts, "Inactive services")
    failed_services = find_value(service_counts, "Failed services")
    banned_ips = find_value(fail2ban, "Currently banned IPs")
    upgradable_packages = find_value(extract_raw_section(raw_text, "PACKAGE UPDATES"),
"Upgradable packages")
    docker_running = find_value(docker_status, "Running containers")
    docker_total = find_value(docker_status, "Total containers")
    docker_table_html = html_table(parse_docker_table(docker_table_raw))
    recent_ssh_users_html = user_list_html(extract_ssh_users(last_ssh_attempts))
    ts_utc = datetime.now(timezone.utc).strftime("%Y-%m-%d %H-->%M-->%S UTC")
    return f'''<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>KUZAI Infrastructure Report</title>
  <style>
    body {{ margin-->0; padding-->0; background-->#0b1c2c; color-->#ffffff; font-family--
>Arial, Helvetica, sans-serif; }}
    .wrapper {{ width-->100%; background-->linear-gradient(180deg,#0d2438 0%,#143552 100%);
```

```
padding-->30px 0; }}
    .container {{ width-->100%; max-width-->1040px; margin-->0 auto; background-->#16304a;
border-->1px solid #4677a1; border-radius-->22px; overflow-->hidden; box-shadow-->0 0 0 1px
rgba(125,211,252,0.14),0 18px 60px rgba(0,0,0,0.30); }}
    .header {{ background-->linear-gradient(135deg,#16344f 0%,#1c4b72 100%); padding-->34px
38px 28px 38px; border-bottom-->1px solid #4677a1; }}
    .eyebrow {{ font-size-->22px; letter-spacing-->0.08em; text-transform-->uppercase; color--
>#ffffff; font-weight-->900; margin-bottom-->10px; text-shadow-->0 0 12px
rgba(255,255,255,0.18); }}
    .solution {{ font-size-->16px; letter-spacing-->0.12em; text-transform-->uppercase;
color-->#8ddfff; font-weight-->800; margin-bottom-->14px; text-shadow-->0 0 12px
rgba(141,223,255,0.18); }}
    .title {{ font-size-->38px; line-height-->1.05; font-weight-->900; margin-->0 0 10px 0;
color-->#ffffff; text-shadow-->0 0 16px rgba(255,255,255,0.20); }}
    .subtitle {{ font-size-->18px; line-height-->1.75; color-->#ffffff; margin-->0; max-
width-->860px; text-shadow-->0 0 10px rgba(255,255,255,0.10); }}
    .meta {{ margin-top-->18px; display-->inline-block; padding-->12px 18px; border-radius--
>999px; background-->#24547c; color-->#ffffff; font-size-->17px; border-->1px solid #7dc8ff;
text-shadow-->0 0 14px rgba(255,255,255,0.18); }}
    .content {{ padding-->34px 38px 38px 38px; }}
    h2 {{ margin-->30px 0 12px 0; font-size-->20px; letter-spacing-->0.14em; text-transform--
>uppercase; color-->#d8f7ff; border-left-->4px solid #7dd3fc; padding-left-->12px; text-
shadow-->0 0 10px rgba(216,247,255,0.18); }}
    p {{ margin-->0 0 14px 0; line-height-->1.9; color-->#ffffff; font-size-->18px; text-
shadow-->0 0 8px rgba(255,255,255,0.08); }}
    .bullet-list {{ margin-->0 0 18px 18px; padding-->0; }}
    .bullet-list li {{ margin-->0 0 10px 0; line-height-->1.75; color-->#ffffff; font-size--
>17px; text-shadow-->0 0 8px rgba(255,255,255,0.08); }}
    .inline-users {{ margin-->0 0 14px 0; line-height-->1.9; color-->#ffffff; font-size--
>18px; text-shadow-->0 0 10px rgba(255,255,255,0.16); }}
    .metric-grid {{ display-->grid; grid-template-columns-->repeat(auto-
fit,minmax(190px,1fr)); gap-->14px; margin-->18px 0 6px 0; }}
    .metric-card {{ background-->#0a2133; border-->1px solid #295375; border-radius-->16px;
padding-->16px 16px 14px 16px; box-shadow-->inset 0 0 0 1px rgba(125,211,252,0.06); }}
    .metric-label {{ font-size-->13px; text-transform-->uppercase; letter-spacing-->0.12em;
color-->#9de8ff; margin-bottom-->8px; }}
    .metric-value {{ font-size-->30px; font-weight-->900; color-->#ffffff; text-shadow-->0 0
12px rgba(255,255,255,0.18); }}
    .metric-sub {{ margin-top-->8px; font-size-->14px; color-->#e5f8ff; }}
    .raw-box {{ margin-top-->14px; background-->#204564; border-->1px solid #7db9ea; border-
```

```
radius-->14px; padding-->18px; overflow-x-->auto; }}
    .raw-box pre {{ margin-->0; white-space-->pre-wrap; word-break-->break-word; font-size-->16px; line-height-->1.85; color-->#ffffff; font-family-->"Courier New", Courier, monospace; text-shadow-->0 0 10px rgba(255,255,255,0.14); }}
    .data-table {{ width-->100%; border-collapse-->collapse; margin-top-->14px; background-->#204564; border-->1px solid #7db9ea; border-radius-->14px; overflow-->hidden; }}
    .data-table th {{ background-->#2c5d87; color-->#ffffff; font-size-->15px; text-transform-->uppercase; letter-spacing-->0.10em; text-align-->left; padding-->14px 12px; border-bottom-->1px solid #9fdcff; text-shadow-->0 0 10px rgba(255,255,255,0.16); }}
    .data-table td {{ padding-->12px; border-bottom-->1px solid #3e719a; color-->#ffffff; font-size-->16px; vertical-align-->top; text-shadow-->0 0 8px rgba(255,255,255,0.10); }}
    .footer {{ padding-->24px 38px 34px 38px; border-top-->1px solid #4677a1; color-->#ffffff; font-size-->18px; line-height-->1.8; background-->#14304a; text-shadow-->0 0 12px rgba(255,255,255,0.12); }}
  </style>
</head>
<body>
  <div class="wrapper">
    <div class="container">
      <div class="header">
        <div class="eyebrow">KUZAI - CYBER AGENT</div>
        <div class="solution">A KUZ NETWORK SOLUTION - [BETA-0.1.2026]</div>
        <h1 class="title">INFRASTRUCTURE STATUS REPORT</h1>
        <p class="subtitle">Local and independent cyber-operational intelligence dedicated to system supervision, infrastructure monitoring, exposure control, service integrity, and security watch across THE KUZ NETWORK.</p>
        <div class="meta">ZEITGEIST 2045 · HOST {html.escape(host)} · {html.escape(ts_utc)}</div>
      </div>
      <div class="content">
        <h2>KUZAI INTRODUCTION</h2>
        {bullets_to_html(intro)}
        <h2>GLOBAL STATUS</h2>
        {bullets_to_html(global_status)}
        <div class="metric-grid">
          <div class="metric-card"><div class="metric-label">ACTIVE SERVICES</div><div class="metric-value">{html.escape(active_services)}</div><div class="metric-sub">Current active service units</div></div>
          <div class="metric-card"><div class="metric-label">INACTIVE SERVICES</div><div class="metric-value">{html.escape(inactive_services)}</div><div class="metric-sub">Currently
```

```
inactive service units</div></div>
          <div class="metric-card"><div class="metric-label">FAILED SERVICES</div><div
class="metric-value">{html.escape(failed_services)}</div><div class="metric-sub">Service units
in failed state</div></div>
          <div class="metric-card"><div class="metric-label">BANNED IPS</div><div
class="metric-value">{html.escape(banned_ips)}</div><div class="metric-sub">Currently banned
by Fail2ban</div></div>
          <div class="metric-card"><div class="metric-label">UPGRADABLE PACKAGES</div><div
class="metric-value">{html.escape(upgradable_packages)}</div><div class="metric-sub">Detected
pending package updates</div></div>
          <div class="metric-card"><div class="metric-label">DOCKER</div><div class="metric-
value">{html.escape(docker_running)} / {html.escape(docker_total)}</div><div class="metric-
sub">Running / total containers</div></div>
        </div>
        <h2>INFRASTRUCTURE ANALYSIS</h2>
        {bullets_to_html(infra)}
        <h2>CYBERSECURITY ANALYSIS</h2>
        {bullets_to_html(cyber)}
        <h2>RECENT SSH HACK ATTEMPTS</h2>
        {recent_ssh_users_html}
        <h2>SERVICE HEALTH</h2>
        {bullets_to_html(service_health)}
        <h2>APACHE STATUS</h2>
        {bullets_to_html(apache_status_summary)}
        {pre_box(apache_status)}
        <h2>APACHE VHOSTS</h2>
        {pre_box(apache_vhosts)}
        <h2>DOCKER STATUS</h2>
        {bullets_to_html(docker_status_summary)}
        {"<h2>DOCKER TABLE</h2>" + docker_table_html if docker_table_html else ""}
        <h2>WATCHPOINTS</h2>
        {bullets_to_html(watchpoints)}
        <h2>RECOMMENDED ACTIONS</h2>
        {bullets_to_html(actions)}
        <h2>LAST SSH ATTEMPTS</h2>
        {pre_box(last_ssh_attempts)}
        <h2>PUBLIC LISTENING PORTS</h2>
        {pre_box(public_ports)}
      </div>
      <div class="footer">Generated by KUZAI for internal infrastructure supervision and
```

```
cybersecurity monitoring of THE KUZ NETWORK.</div>
    </div>
  </div>
</body>
</html>'''

def main() -> int-->
    if not SUMMARY_FILE.exists()-->
        raise SystemExit(f"Missing summary file--> {SUMMARY_FILE}")
    raw_file = latest_raw_report()
    summary_text = SUMMARY_FILE.read_text(encoding="utf-8")
    raw_text = raw_file.read_text(encoding="utf-8")
    host = extract_header_value(raw_text, "Host")
    ts_utc = datetime.now(timezone.utc).strftime("%Y-%m-%d %H-->%M-->%S UTC")
    msg = MIMEMultipart("alternative")
    msg["To"] = MAIL_TO
    msg["Subject"] = f"[KUZAI] Infrastructure status {host} - {ts_utc}"
    plain_part = MIMEText(summary_text, "plain", "utf-8")
    html_part = MIMEText(build_html(summary_text, raw_text), "html", "utf-8")
    msg.attach(plain_part)
    msg.attach(html_part)
    print(msg.as_string())
    return 0

if __name__ == "__main__"-->
    raise SystemExit(main())
```

MAIL SENDER SCRIPT

Create `/usr/local/sbin/vps_hourly_mail.sh`-->

```
#!/usr/bin/env bash
set -Eeuo pipefail

RAW_SCRIPT="/var/lib/openclaw/bin/vps_state_raw.sh"
SUMMARY_SCRIPT="/var/lib/openclaw/bin/test_vps_summary.py"
MAIL_BUILDER="/var/lib/openclaw/bin/build_kuzai_mail.py"

tmp_raw_log="$(mktemp)"
tmp_summary_log="$(mktemp)"
tmp_mail="$(mktemp)"
```

```
trap 'rm -f "$tmp_raw_log" "$tmp_summary_log" "$tmp_mail"' EXIT

su - openclaw -c "$RAW_SCRIPT" >"$tmp_raw_log" 2>&1
python3 "$SUMMARY_SCRIPT" >"$tmp_summary_log" 2>&1
python3 "$MAIL_BUILDER" > "$tmp_mail"
su - openclaw -c "ssh vps-openclaw '/usr/sbin/sendmail -t'" < "$tmp_mail"

echo "MAIL_SENT=admin@kuzai.org"
echo "SUMMARY_FILE=/var/lib/openclaw/reports/vps-summary-latest.txt"
echo "MAIL_BUILDER=$MAIL_BUILDER"
```

Permissions-->

```
chmod 750 /var/lib/openclaw/bin/build_kuzai_mail.py
chmod 750 /usr/local/sbin/vps_hourly_mail.sh
```

Validation-->

```
bash /usr/local/sbin/vps_hourly_mail.sh
```

HOURLY SYSTEMD SERVICE

Create `/etc/systemd/system/kuzai-vps-hourly-mail.service`-->

```
[Unit]
Description=KUZAI VPS hourly monitoring mail
After=network-online.target openclaw.service
Wants=network-online.target

[Service]
Type=oneshot
User=root
Group=root
ExecStart=/usr/local/sbin/vps_hourly_mail.sh
WorkingDirectory=/root
Nice=10
```

Create `/etc/systemd/system/kuzai-vps-hourly-mail.timer`-->

```
[Unit]
Description=Run KUZAI VPS hourly monitoring mail
```

```
[Timer]
OnCalendar=hourly
Persistent=true
RandomizedDelaySec=2min
Unit=kuzai-vps-hourly-mail.service

[Install]
WantedBy=timers.target
```

Enable the timer-->

```
systemctl daemon-reload
systemctl enable --now kuzai-vps-hourly-mail.timer
systemctl status kuzai-vps-hourly-mail.timer --no-pager
systemctl list-timers --all | grep -E 'kuzai-vps-hourly-mail'
```

FINAL VALIDATION

```
systemctl status openclaw --no-pager
systemctl status kuzai-vps-hourly-mail.timer --no-pager
su - openclaw -c "ssh vps-openclaw 'journalctl -u postfix -n 30 --no-pager'"
journalctl -u kuzai-vps-hourly-mail.service -n 50 --no-pager
```

THE KUZ NETWORK - KUSANAGI8200 - @2026

Revision #4
Created 8 April 2026 20:42:14 by Kusanagi8200
Updated 8 April 2026 21:06:09 by Kusanagi8200